

Data & AI

6. Transformations and Backpropagation

Dasaem Jeong · Art & Technology, Sogang University

Last Time

One layer is a matrix multiplication

- Each column of W holds one neuron's weights
- Run trains every weight and every bias with the rule from lecture 4

Today

jdassam.github.io/mas1004/demos/lecture06-one-layer-2d · **lecture06-backprop-by-hand**

- What a matrix does to the plane, and what the bias and the activation add
- How a trained network moves points, layer by layer
- Backpropagation: every slope from one pass backward through the loss

What a matrix does to the plane

A Point as Input

A layer of two neurons, two inputs

$$\begin{pmatrix} x & y \end{pmatrix} \times \begin{pmatrix} w_{11} & w_{12} \\ w_{21} & w_{22} \end{pmatrix} = \begin{pmatrix} x \cdot w_{11} + y \cdot w_{21} & x \cdot w_{12} + y \cdot w_{22} \end{pmatrix} = \begin{pmatrix} h_1 & h_2 \end{pmatrix}$$

- The input is a point $p = (x, y)$, and the output is another point (h_1, h_2)
- Same rule as lecture 5: column 1 is neuron 1, column 2 is neuron 2

Where $(1, 0)$ and $(0, 1)$ Go

Demo, preset “Rotate 30° ”

- $(1, 0) \times W$ is the first row of W , and $(0, 1) \times W$ is the second row
- The two arrows tell where every other point goes

Three More Presets

Demo, presets “Stretch”, “Shear”, “Flip”

Stretch: $W = \begin{bmatrix} 2 & 0 \\ 0 & 1 \end{bmatrix}$

Shear: $W = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix}$

Flip: $W = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$

- Grid lines stay straight and parallel, and the origin stays put

Matrix Multiplication as a Transformation

3Blue1Brown, Essence of Linear Algebra, Chapter 3

Video: youtu.be/kYB8lZa5AuE

- A matrix moves every point of the plane at once: rotate, stretch, shear
- The video uses column vectors: the matrix's columns are where $(1, 0)$ and $(0, 1)$ go

Adding the Bias

Demo, “+ b” on

- The whole plane moves by (b_1, b_2) , and the origin moves with it

ReLU on the Plane

Demo, “act” on, ReLU

- ReLU sends every negative coordinate to 0, which folds the plane: the first step that bends a straight line

tanh on the Plane

Demo, “act” on, tanh

- Every coordinate is pressed into $(-1, 1)$, more strongly the farther it is from 0

ReLU Applied to the Plane

vcubingx, The Neural Network, A Visual Introduction

Video: youtu.be/UOvPeC8WOt8

- $h = (Wx + b)^+$: multiply by W , add b , then ReLU (the $+$)

A Random Matrix

Demo, preset “Random”

- Read the two rows of W and say where $(1, 0)$ and $(0, 1)$ go before you press Play
- Then turn on ReLU and say which part of the F gets folded

A trained network, layer by layer

Layer 1 of a Trained Network

Layer by Layer in 2D · XOR · rotate 31.0° clockwise

shot

- The same steps as the demo, with W and b found by gradient descent

Layer 1 of a Trained Network

**Layer by Layer in 2D · XOR · stretch $\times 2.17$ along x,
 $\times 1.28$ along y**

shot

- The same steps as the demo, with W and b found by gradient descent

Layer 1 of a Trained Network

**Layer by Layer in 2D · XOR · rotate 42.8°
counterclockwise**

shot

- The same steps as the demo, with W and b found by gradient descent

Layer 1 of a Trained Network

Layer by Layer in 2D · XOR · shift by $(-0.11, 0.03)$

shot

- The same steps as the demo, with W and b found by gradient descent

Layer 1 of a Trained Network

Layer by Layer in 2D · XOR · tanh: squash into $(-1, 1)$

shot

- The same steps as the demo, with W and b found by gradient descent

The Plane After Every Layer

Layer by Layer in 2D · thumbnail strip

- Each layer rotates, stretches, shifts and squashes the plane again
- In the last space one straight line separates the two colors

Crumpled Paper

François Chollet, Deep Learning with Python (2017)

Photo: G. Chen, CMU 95-865 lecture slides

- Two sheets of paper crumpled into one ball
- Layer by layer, the network uncrumples the ball until the sheets come apart

Backpropagation

Measuring Slopes in Lecture 4

Nudge one knob by Δ , compute the loss again

Knobs	Loss computations per step
8 (lecture 4, eight bumps)	9
33 (Neuron Patch, “2 layers”)	34
10^{11} (a large language model)	about 10^{11}

- One computation for the current loss, one more for every nudged knob

The Loss as a Chain of Small Operations

Backprop by Hand, preset “1 neuron”, point $(x, y) = (2, 3)$

$$x \rightarrow \times w \rightarrow + b \rightarrow \hat{y} \rightarrow - y \rightarrow e \rightarrow ^2 \rightarrow \text{loss}$$

- Four operations: multiply, add, subtract y , square
- Loss for one point: $(\hat{y} - y)^2$

Forward Pass

Backprop by Hand, preset “1 neuron”

shot

- $w \cdot x = -1, \hat{y} = -1, e = \hat{y} - y = -4, \text{loss} = 16$

Backward Pass

Backprop by Hand, preset “1 neuron”

shot

- Start at the loss with slope 1, then move one box to the left at a time
- w gets -16 : nudge w by 0.001 and the loss changes by about -0.016

Four Rules

The slope coming from the right, g , is passed to the left

Operation	Rule	Example
$\times$	multiply g by the other input	$\hat{y} = w \cdot h$, $h = 2$, $g = -6$: w gets $2 \times (-6) = -12$
$+$	pass g unchanged	$z = w \cdot x + b$: b gets g
ReLU	pass g if $z > 0$, otherwise 0	$z = -2$: the slope stops here
2	multiply g by $2e$	$e = -4$, $g = 1$: e gets -8

One ReLU Neuron

Backprop by Hand, preset “1 ReLU”, point (2, 3)

shot

- Forward: $z_1 = 2$, $h_1 = 2$, $\hat{y} = 0$, $e = -3$, $\text{loss} = 9$
- Backward: $\hat{y} -6$, $w_2 -12$, $b_2 -6$, $h_1 3$, $z_1 3$, $w_1 6$, $b_1 3$

A Closed ReLU

Backprop by Hand, preset “3 ReLU”, point (2, 3)

shot

- N2 has $z_2 = -2$, so ReLU passes 0: w_2 and b_2 get slope 0
- v_2 also gets 0, because its other input h_2 is 0

Nudge and Backprop

Backprop by Hand, “Nudge each knob”

shot

- Nudge: one loss computation per knob, plus one
- Backprop: one pass forward, one pass backward, whatever the number of knobs

Take a Step

Backprop by Hand, $w \leftarrow w - lr \times \text{slope}$, lr 0.02

shot

- Every knob moves against its slope at once: the loss for this point drops from 9 to 5.39
- Run in Neuron Patch is this step, repeated

From One Point to 120 Points

From $(\hat{y} - y)^2$ to the mean over 120 points

- Neuron Patch's loss is the mean of this one-point loss over all 120 points
- The slope of a mean is the mean of the slopes: run the same pass for each point and average

What we have

Three Parts of Machine Learning

- A model: layers that rotate, stretch, shift and fold the input space
- A loss: one number that measures how far the predictions are from the data
- A search: gradient descent, with every slope from one backward pass

Coming Up

- Week 4: working with LLMs, reading and verifying generated results
- Sep 24 (Thu): Chuseok holiday, no class

What changes when the network has 10^{11} knobs?